

13. IBM Watson and Cognitive Computing

Objectives

In this chapter, you'll:

- See Watson's range of services and use their Lite tier to become familiar with them at no charge.
- Try lots of demos of Watson services.
- Understand what cognitive computing is and how you can incorporate it into your applications.
- Register for an IBM Cloud account and get credentials to use various services.
- Install the Watson Developer Cloud Python SDK to interact with Watson services.
- Develop a traveler's companion language translator app by using Python to weave together a mashup of the Watson Speech to Text, Language Translator and Text to Speech services.
- Check out additional resources, such as IBM Watson Redbooks that will help you jump start your custom Watson application development.

Outline

13.1 Introduction: IBM Watson and Cognitive Computing

13.2 IBM Cloud Account and Cloud Console

13.3 Watson Services

13.4 Additional Services and Tools

[3.5 Watson Developer Cloud Python SDK](#)

[3.6 Case Study: Traveler's Companion Translation App](#)

[3.6.1 Before You Run the App](#)

[3.6.2 Test-Driving the App](#)

[3.6.3 SimpleLanguageTranslator.py Script Walkthrough](#)

[3.7 Watson Resources](#)

[3.8 Wrap-Up](#)

13.1 INTRODUCTION: IBM WATSON AND COGNITIVE COMPUTING

In chapter 1, we discussed some key IBM artificial-intelligence accomplishments, including beating the two best human Jeopardy! players in a \$1 million match. Watson won the competition and IBM donated the prize money to charity. Watson simultaneously executed hundreds of language-analysis algorithms to locate correct answers in 200 million pages of content (including all of Wikipedia) requiring four terabytes of storage.^{1, 2} IBM researchers trained Watson using machine-learning and reinforcement-learning techniques—we discuss machine learning in the next chapter.³

¹ <https://www.techrepublic.com/article/ibm-watson-the-inside-story-of-how-the-jeopardy--winning-supercomputer-was-born-and-what-it-wants-to-do-next/>.

² [https://en.wikipedia.org/wiki/Watson_\(computer\)](https://en.wikipedia.org/wiki/Watson_(computer)).

³ <https://www.aaai.org/Magazine/Watson/watson.php>, *AI Magazine*, Fall 2010.

Early in our research for this book, we recognized the rapidly growing importance of Watson, so we placed Google Alerts on Watson and related topics. Through those alerts and the newsletters and blogs we follow, we accumulated 900+ current Watson-related articles, documentation pieces and videos. We investigated many competitive services and found Watson's "no credit card required" policy and free *Lite tier* services⁴ to be among friendliest to people who'd like to experiment with Watson's services at no

harge.

⁴ Always check the latest terms on IBMs website as the terms and services may change.

IBM Watson is a cloud-based cognitive-computing platform being employed across a wide range of real-world scenarios. Cognitive-computing systems simulate the pattern-recognition and decision-making capabilities of the human brain to “learn” as they consume more data. ⁵ , ⁶ , ⁷ We overview Watson’s broad range of web services and provide a hands-on Watson treatment, demonstrating many Watson capabilities. The table on the next page shows just a few of the ways in which organizations are using Watson.

⁵ <http://whatis.techtarget.com/definition/cognitive-computing>.

⁶ https://en.wikipedia.org/wiki/Cognitive_computing.

⁷ <https://www.forbes.com/sites/bernardmarr/2016/03/23/what-everyone-should-know-about-cognitive-computing>.

Watson offers an intriguing set of capabilities that you can incorporate into your applications. In this chapter, you’ll set up an IBM Cloud account ⁸ and use the *Lite tier* and IBM’s Watson demos to experiment with various web services, such as natural language translation, speech-to-text, text-to-speech, natural language understanding, chatbots, analyzing- text for tone and visual object recognition in images and video. We’ll briefly overview some additional Watson services and tools.

⁸ IBM Cloud previously was called Bluemix. Youll still see `bluemix` in many of this chapters URLs.

Watson use cases

ad targeting	fraud prevention	
artificial intelligence	game playing	personal assistants
	genetics	predictive maintenance
augmented intelligence	healthcare	product recommendations
augmented reality	image processing	

chatbots	IoT (Internet of Things)	robots and drones
closed captioning		self-driving cars
	language translation	
cognitive computing	machine learning	sentiment and mood analysis
conversational interfaces	malware detection	smart homes
crime prevention	medical diagnosis and treatment	sports
customer support	medical imaging	supply-chain management
detecting cyberbullying	music	threat detection
drug development	natural language processing	virtual reality
education	natural language understanding	voice analysis
facial recognition	object recognition	weather forecasting
finance		workplace safety

You'll install the Watson Developer Cloud Python Software Development Kit (SDK) for programmatic access to Watson services from your Python code. Then, in our hands-on implementation case study, you'll develop a traveler's companion translation app by quickly and conveniently *mashing up* several Watson services. The app enables English-only and Spanish-only speakers to communicate with one another verbally, despite the language barrier. You'll transcribe English and Spanish audio recordings to text, translate the text to the other language, then synthesize and play English and Spanish audio from the translated text.

Watson is a dynamic and evolving set of capabilities. During the time we worked on this book, new services were added and existing services were updated and/or removed multiple times. The descriptions of the Watson services and the steps we present were accurate as of the time of this writing. We'll post updates as necessary on the book's web page at www.deitel.com.

3.2 IBM CLOUD ACCOUNT AND CLOUD CONSOLE

You'll need a free IBM Cloud account to access Watson's Lite tier services. Each service's description web page lists the service's tiered offerings and what you get with each tier. Though the Lite tier services limit your use, they typically offer what you'll need to familiarize yourself with Watson features and begin using them to develop apps. The limits are subject to change, so rather than list them here, we point you to each service's web page. IBM increased the limits significantly on some services while we were writing this book. Paid tiers are available for use in commercial-grade applications.

To get a free IBM Cloud account, follow the instructions at:

<https://console.bluemix.net/docs/services/watson/index.html#about>

You'll receive an e-mail. Follow its instructions to confirm your account. Then you can log in to the IBM Cloud console. Once there, you can go to the **Watson dashboard** at:

<https://console.bluemix.net/developer/watson/dashboard>

where you can:

- Browse the Watson services.
- Link to the services you've already registered to use.
- Look at the developer resources, including the Watson documentation, SDKs and various resources for learning Watson.
- View the apps you've created with Watson.

Later, you'll register for and get your credentials to use various Watson services. You can view and manage your list of services and your credentials in the **IBM Cloud dashboard** at:

<https://console.bluemix.net/dashboard/apps>

You can also click **Existing Services** in the Watson dashboard to get to this list.

13.3 WATSON SERVICES

This section overviews many of Watson's services and provides links to the details for

ach. Be sure to run the demos to see the services in action. For links to each Watson service's documentation and API reference, visit:

<https://console.bluemix.net/developer/watson/documentation>

We provide footnotes with links to each service's details. When you're ready to use a particular service, click the **Create** button on its details page to set up your credentials.

Watson Assistant

The **Watson Assistant service**⁹ helps you build chatbots and virtual assistants that enable users to interact via natural language text. IBM provides a web interface that you can use to *train* the Watson Assistant service for specific scenarios associated with your app. For example, a weather chatbot could be trained to respond to questions like, “What is the weather forecast for New York City?” In a customer service scenario, you could create chatbots that answer customer questions and route customers to the correct department, if necessary. Try the demo at the following site to see some sample interactions:

⁹ <https://console.bluemix.net/catalog/services/watson-assistant-formerly-conversation>.

<https://www.ibm.com/watson/services/conversation/demo/index.html#demo>

Visual Recognition

The **Visual Recognition service**¹⁰ enables apps to locate and understand information in images and video, including colors, objects, faces, text, food and inappropriate content. IBM provides predefined models (used in the service's demo), or you can train and use your own (as you'll do in the “Deep Learning” chapter). Try the following demo with the images provided and upload some of your own:

¹⁰ <https://console.bluemix.net/catalog/services/visual-recognition>.

<https://watson-visual-recognition-duo-dev.ng.bluemix.net/>

Speech to Text

The **Speech to Text service**,¹¹ which we'll use in building this chapter's app, converts speech audio files to text transcriptions of the audio. You can give the service keywords to “listen” for, and it tells you whether it found them, what the likelihood of a match

as and where the match occurred in the audio. The service can distinguish among multiple speakers. You could use this service to help implement voice-controlled apps, transcribe live audio and more. Try the following demo with its sample audio clips or upload your own:

⁻¹ <https://console.bluemix.net/catalog/services/speech-to-text>.

<https://speech-to-text-demo.ng.bluemix.net/>

Text to Speech

The **Text to Speech service**,² which we'll also use in building this chapter's app, enables you to synthesize speech from text. You can use **Speech Synthesis Markup Language (SSML)** to embed instructions in the text for control over voice inflection, cadence, pitch and more. Currently, this service supports English (U.S. and U.K.), French, German, Italian, Spanish, Portuguese and Japanese. Try the following demo with its plain sample text, its sample text that includes SSML and text that you provide:

⁻² <https://console.bluemix.net/catalog/services/text-to-speech>.

<https://text-to-speech-demo.ng.bluemix.net/>

Language Translator

The **Language Translator service**,³ which we'll also use in building in this chapter's app, has two key components:

⁻³ <https://console.bluemix.net/catalog/services/language-translator>.

- translating text between languages and
- identifying text as being written in one of over 60 languages.

Translation is supported to and from English and many languages, as well as between other languages. Try translating text into various languages with the following demo:

<https://language-translator-demo.ng.bluemix.net/>

Natural Language Understanding

The **Natural Language Understanding service**⁴ analyzes text and produces

nformation including the text's overall sentiment and emotion and keywords ranked by their relevance. Among other things, the service can identify

⁴ <https://console.bluemix.net/catalog/services/natural-language-understanding>.

- people, places, job titles, organizations, companies and quantities.
- categories and concepts like sports, government and politics.
- parts of speech like subjects and verbs.

You also can train the service for industry- and application-specific domains with Watson Knowledge Studio (discussed shortly). Try the following demo with its sample text, with text that you paste in or by providing a link to an article or document online:

<https://natural-language-understanding-demo.ng.bluemix.net/>

Discovery

The **Watson Discovery service**⁵ shares many features with the Natural Language Understanding service but also enables enterprises to store and manage documents. So, for example, organizations can use Watson Discovery to store all their text documents and be able to use natural language understanding across the entire collection. Try this service's demo, which enables you to search recent news articles for companies:

⁵ <https://console.bluemix.net/catalog/services/discovery>.

<https://discovery-news-demo.ng.bluemix.net/>

Personality Insights

The **Personality Insights service**⁶ analyzes text for personality traits. According to the service description, it can help you “gain insight into how and why people think, act, and feel the way they do. This service applies linguistic analytics and personality theory to infer attributes from a person's unstructured text.” This information could be used to target product advertising at the people most likely to purchase those products. Try the following demo with tweets from various Twitter accounts or documents built into the demo, with text documents that you paste into the demo or with your own Twitter account:

⁶ <https://console.bluemix.net/catalog/services/personality->

nsights.

<https://personality-insights-livedemo.ng.bluemix.net/>

Tone Analyzer

The **Tone Analyzer service**⁷ analyzes text for its tone in three categories:

⁷ <https://console.bluemix.net/catalog/services/tone-analyzer>.

- emotions—anger, disgust, fear, joy, sadness.
- social propensities—openness, conscientiousness, extroversion, agreeableness and emotional range.
- language style—analytical, confident, tentative.

Try the following demo with sample tweets, a sample product review, a sample e-mail or text you provide. You'll see the tone analyses at both the document and sentence levels:

<https://tone-analyzer-demo.ng.bluemix.net/>

Natural Language Classifier

You *train* the **Natural Language Classifier service**⁸ with sentences and phrases that are specific to your application and classify each sentence or phrase. For example, you might classify “I need help with your product” as “tech support” and “My bill is incorrect” as “billing.” Once you’ve trained your classifier, the service can receive sentences and phrases, then use Watson’s cognitive computing capabilities and your classifier to return the best matching classifications and their match probabilities. You might then use the returned classifications and probabilities to determine the next steps in your app. For example, in a customer service app where someone is calling in with a question about a particular product, you might use Speech to Text to convert a question into text, use the Natural Language Classifier service to classify the text, then route the call to the appropriate person or department. This service *does not offer a Lite tier*. In the following demo, enter a question about the weather—the service will respond by indicating whether your question was about the temperature or the weather conditions:

⁸ <https://console.bluemix.net/catalog/services/natural-language->

lassifier.

<https://natural-language-classifier-demo.ng.bluemix.net/>

Synchronous and Asynchronous Capabilities

Many of the APIs we discuss throughout the book are **synchronous**—when you call a function or method, the program *waits* for the function or method to return before moving on to the next task. **Asynchronous** programs can start a task, continue doing other things, then be *notified* when the original task completes and returns its results. Many Watson services offer both synchronous and asynchronous APIs.

The Speech to Text demo is a good example of asynchronous APIs. The demo processes sample audio of two people speaking. As the service transcribes the audio, it returns intermediate transcription results, even if it has not yet been able to distinguish among the speakers. The demo displays these intermediate results in parallel with the service’s continued work. Sometimes the demo displays “Detecting speakers” while the service figures out who is speaking. Eventually, the service sends updated transcription results for distinguishing among the speakers, and the demo then replaces the prior transcription results.

With today’s multi-core computers and multi-computer clusters, the asynchronous APIs can help you improve program performance. However, programming with them can be more complicated than programming with synchronous APIs. When we discuss installing the Watson Developer Cloud Python SDK, we provide a link to the SDK’s code examples on GitHub, where you can see examples that use synchronous and asynchronous versions of several services. Each service’s API reference provides the complete details.

13.4 ADDITIONAL SERVICES AND TOOLS

In this section, we overview several Watson advanced services and tools.

Watson Studio

Watson Studio⁹ is the new Watson interface for creating and managing your Watson projects and for collaborating with your team members on those projects. You can add data, prepare your data for analysis, create Jupyter Notebooks for interacting with your data, create and train models and work with Watson’s deep-learning capabilities.

Watson Studio offers a single-user Lite tier. Once you’ve set up your Watson Studio Lite access by clicking **Create** on the service’s details web page

⁹ <https://console.bluemix.net/catalog/services/data-science-experience>.

<https://console.bluemix.net/catalog/services/data-science-experience>

you can access Watson Studio at

<https://dataplatform.cloud.ibm.com/>

Watson Studio contains preconfigured projects.¹⁰ Click **Create a project** to view them:

¹⁰ <https://dataplatform.cloud.ibm.com/>.

- Standard—“Work with any type of asset. Add services for analytical assets as you need them.”
- Data Science—“Analyze data to discover insights and share your findings with others.”
- Visual Recognition—“Tag and classify visual content using the Watson Visual Recognition service.”
- Deep Learning—“Build neural networks and deploy deep learning models.”
- Modeler—“Build modeler flows to train SPSS models or design deep neural networks.”
- Business Analytics—“Create visual dashboards from your data to gain insights faster.”
- Data Engineering—“Combine, cleanse, analyze, and shape data using Data Refinery.”
- Streams Flow—“Ingest and analyze streaming data using the Streaming Analytics service.”

Knowledge Studio

Various Watson services work with *predefined* models, but also allow you to provide custom models that are trained for specific industries or applications. Watson’s **Knowledge Studio**¹¹ helps you build custom models. It allows enterprise teams to

ork together to create and train new models, which can then be deployed for use by Watson services.

¹ <https://console.bluemix.net/catalog/services/knowledge-studio>.

Machine Learning

The **Watson Machine Learning service**² enables you to add predictive capabilities to your apps via popular machine-learning frameworks, including Tensorflow, Keras, scikit-learn and others. You'll use scikit-learn and Keras in the next two chapters.

² <https://console.bluemix.net/catalog/services/machine-learning>.

Knowledge Catalog

The **Watson Knowledge Catalog**^{3,4} is an advanced enterprise-level tool for securely managing, finding and sharing your organization's data. The tool offers:

³ <https://medium.com/ibm-watson/introducing-ibm-watson-knowledge-catalog-cf42c13032c1>.

⁴ <https://dataplatform.cloud.ibm.com/docs/content/catalog/overview-kc.html>.

- Central access to an enterprise's local and cloud-based data and machine learning models.
- Watson Studio support so users can find and access data, then easily use it in machine-learning projects.
- Security policies that ensure only the people who should have access to specific data actually do.
- Support for over 100 data cleaning and wrangling operations.
- And more.

Cognos Analytics

The IBM **Cognos Analytics**⁵ service, which has a 30-day free trial, uses AI and machine learning to discover and visualize information in your data, without any programming on your part. It also provides a natural-language interface that enables you to ask questions which Cognos Analytics answers based on the knowledge it gathers

from your data.

⁵ <https://www.ibm.com/products/cognos-analytics>.

13.5 WATSON DEVELOPER CLOUD PYTHON SDK

In this section, you'll install the modules required for the next section's full-implementation Watson case study. For your coding convenience, IBM provides the **Watson Developer Cloud Python SDK** (software development kit). Its `watson_developer_cloud module` contains classes that you'll use to interact with Watson services. You'll create objects for each service you need, then interact with the service by calling the object's methods.

To install the SDK⁶ open an Anaconda Prompt (Windows; open as Administrator), Terminal (macOS/Linux) or shell (Linux), then execute the following command⁷:

⁶For detailed installation instructions and troubleshooting tips, see <https://github.com/watson-developer-cloud/python-sdk/blob/develop/README.md>.

⁷Windows users might need to install Microsofts C++ build tools from <https://visualstudio.microsoft.com/visual-cpp-build-tools/>, then install the `watson-developer-cloud` module.

```
pip install --upgrade watson-developer-cloud
```

Modules We'll Need for Audio Recording and Playback

You'll also need two additional modules for audio recording (PyAudio) and playback (PyDub). To install these, use the following commands⁸:

⁸Mac users might need to first execute `conda install -c conda-forge portaudio`.

```
pip install pyaudio
pip install pydub
```

SDK Examples

On GitHub, IBM provides sample code demonstrating how to access Watson services using the Watson Developer Cloud Python SDK's classes. You can find the examples at:

13.6 CASE STUDY: TRAVELER'S COMPANION TRANSLATION APP

Suppose you're traveling in a Spanish-speaking country, but you do not speak Spanish, and you need to communicate with someone who does not speak English. You could use a translation app to speak in English, and the app could translate that, then speak it in Spanish. The Spanish-speaking person could then respond, and the app could translate that and speak it to you in English.

Here, you'll use three powerful IBM Watson services to implement such a traveler's companion translation app,⁹ enabling people who speak different languages to converse in near real time. Combining services like this is known as creating a **mashup**. This app also uses simple file-processing capabilities that we introduced in the "Files and Exceptions" chapter.

⁹These services could change in the future. If they do, we'll post updates on the books web page at <http://www.deitel.com/books/IntroToPython>.

13.6.1 Before You Run the App

You'll build this app using the Lite (free) tiers of several IBM Watson services. Before executing the app, make sure that you've registered for an IBM Cloud account, as we discussed earlier in the chapter, so you can get credentials for each of the three services the app uses. Once you have your credentials (described below), you'll insert them in our `keys.py` file (located in the `ch13` examples folder) that we import into the example. Never share your credentials.

As you configure the services below, each service's credentials page also shows you the service's URL. These are the default URLs used by the Watson Developer Cloud Python SDK, so you do not need to copy them. In [section 13.6.3](#), we present the `SimpleLanguageTranslator.py` script and a detailed walkthrough of the code.

Registering for the Speech to Text Service

This app uses the Watson Speech to Text service to transcribe English and Spanish audio files to English and Spanish text, respectively. To interact with the service, you must get a username and password. To do so:

1. **Create a Service Instance:** Go to

`https://console.bluemix.net/catalog/services-/speech-to-text`
and click the **Create** button on the bottom of the page. This auto-generates an API key for you and takes you to a tutorial for working with the Speech to Text service.

2. **Get Your Service Credentials:** To see your API key, click **Manage** at the top-left of the page. To the right of **Credentials**, click **Show credentials**, then copy the **API Key**, and paste it into the variable `speech_to_text_key`'s string in the `keys.py` file provided in this chapter's `ch13` examples folder.

Registering for the Text to Speech Service

In this app, you'll use the Watson Text to Speech service to synthesize speech from text. This service also requires you to get a username and password. To do so:

1. **Create a Service Instance:** Go to `https://console.bluemix.net/catalog/services/text-to-speech` and click the **Create** button on the bottom of the page. This auto-generates an API key for you and takes you to a tutorial for working with the Text to Speech service.
2. **Get Your Service Credentials:** To see your API key, click **Manage** at the top-left of the page. To the right of **Credentials**, click **Show credentials**, then copy the **API Key** and paste it into the variable `text_to_speech_key`'s string in the `keys.py` file provided in this chapter's `ch13` examples folder.

Registering for the Language Translator Service

In this app, you'll use the Watson Language Translator service to pass text to Watson and receive back the text translated into another language. This service requires you to get an API key. To do so:

1. **Create a Service Instance:** Go to `https://console.bluemix.net/catalog/services-/language-
translator` and click the **Create** button on the bottom of the page. This auto-generates an API key for you and takes you to a page to manage your instance of the service.
2. **Get Your Service Credentials:** To the right of **Credentials**, click **Show credentials**, then copy the **API Key** and paste it into the variable `translate_key`'s string in the `keys.py` file provided in this chapter's `ch13` examples folder.

Retrieving Your Credentials

To view your credentials at any time, click the appropriate service instance at:

<https://console.bluemix.net/dashboard/apps>

13.6.2 Test-Driving the App

Once you've added your credentials to the script, open an Anaconda Prompt (Windows), a Terminal (macOS/Linux) or a shell (Linux). Run the script⁹ by executing the following command from the `ch13` examples folder:

⁹The `pydub.playback` module we use in this app issues a warning when you run our script. The warning has to do with module features we don't use and can be ignored. To eliminate this warning, you can install `ffmpeg` for Windows, macOS or Linux from <https://www.ffmpeg.org>.

```
ipython SimpleLanguageTranslator.py
```

Processing the Question

The app performs 10 steps, which we point out via comments in the code. When the app begins executing:

Step 1 prompts for and records a question. First, the app displays:

```
Press Enter then ask your question in English
```

and waits for you to press *Enter*. When you do, the app displays:

```
Recording 5 seconds of audio
```

Speak your question. We said, "Where is the closest bathroom?" After five seconds, the app displays:

```
Recording complete
```

Step 2 interacts with Watson's Speech to Text service to transcribe your audio to text and displays the result:

```
English: where is the closest bathroom
```

Step 3 then uses Watson’s Language Translator service to translate the English text to Spanish and displays the translated text returned by Watson:

```
Spanish: ¿Dónde está el baño más cercano?
```

Step 4 passes this Spanish text to Watson’s Text to Speech service to convert the text to an audio file.

Step 5 plays the resulting Spanish audio file.

Processing the Response

At this point, we’re ready to process the Spanish speaker’s response.

Step 6 displays:

```
Press Enter then speak the Spanish answer
```

and waits for you to press *Enter*. When you do, the app displays:

```
Recording 5 seconds of audio
```

and the Spanish speaker records a response. We do not speak Spanish, so we used Watson’s Text to Speech service to *prerecord* Watson saying the Spanish response “El baño más cercano está en el restaurante,” then played that audio loud enough for our computer’s microphone to record it. We provided this prerecorded audio for you as `SpokenResponse.wav` in the `ch13` folder. If you use this file, play it quickly after pressing *Enter* above as the app records for only 5 seconds.¹ To ensure that the audio loads and plays quickly, you might want to play it once before you press *Enter* to begin recording. After five seconds, the app displays:

¹For simplicity, we set the app to record five seconds of audio. You can control the duration with the variable `SECONDS` in function `record_audio`. It’s possible to create a recorder that begins recording once it detects sound and stops recording after a period of silence, but the code is more complicated.

```
Recording complete
```

Step 7 interacts with Watson’s Speech to Text service to transcribe the Spanish audio to text and displays the result:

```
Spanish response: el baño más cercano está en el restaurante
```

Step 8 then uses Watson’s Language Translator service to translate the Spanish text to English and displays the result:

```
English response: The nearest bathroom is in the restaurant
```

Step 9 passes the English text to Watson’s Text to Speech service to convert the text to an audio file.

Step 10 then plays the resulting English audio.

13.6.3 SimpleLanguageTranslator.py Script Walkthrough

In this section, we present the SimpleLanguageTranslator.py script’s source code, which we’ve divided into small consecutively numbered pieces. Let’s use a top-down approach as we did in the “Control Statements” chapter. Here’s the top:

Create a translator app that enables English and Spanish speakers to communicate.

The first refinement is:

Translate a question spoken in English into Spanish speech.

Translate the answer spoken in Spanish into English speech.

We can break the first line of the second refinement into five steps:

Step 1: Prompt for then record English speech into an audio file.

Step 2: Transcribe the English speech to English text.

Step 3: Translate the English text into Spanish text.

Step 4: Synthesize the Spanish text into Spanish speech and save it into an audio file.

Step 5: Play the Spanish audio file.

e can break the second line of the second refinement into five steps:

Step 6: Prompt for then record Spanish speech into an audio file.

Step 7: Transcribe the Spanish speech to Spanish text.

Step 8: Translate the Spanish text into English text.

Step 9: Synthesize the English text into English speech and save it into an audio file.

Step 10: Play the English audio.

This top-down development makes the benefits of the divide-and-conquer approach clear, focusing our attention on small pieces of a more significant problem.

In this section's script, we implement the 10 steps specified in the second refinement.

Steps 2 and **7** use the Watson Speech to Text service, **Steps 3** and **8** use the Watson Language Translator service, and **Steps 4** and **9** use the Watson Text to Speech service.

Importing Watson SDK Classes

Lines 4–6 import classes from the `watson_developer_cloud` module that was installed with the Watson Developer Cloud Python SDK. Each of these classes uses the Watson credentials you obtained earlier to interact with a corresponding Watson service:

- Class **SpeechToTextV1**² enables you to pass an audio file to the Watson Speech to Text service and receive a JSON³ document containing the text transcription.

²The `v1` in the class name indicates the services version number. As IBM revises its services, it adds new classes to the `watson_developer_cloud` module, rather than modifying the existing classes. This ensures that existing apps do not break when the services are updated. The Speech to Text and Text to Speech services are each Version 1 (`v1`) and the Language Translator service is Version 3 (`v3`) at the time of this writing.

³We introduced JSON in the previous chapter, Data Mining Twitter.

- Class **LanguageTranslatorV3** enables you to pass text to the Watson Language Translator service and receive a JSON document containing the translated text.
- Class **TextToSpeechV1** enables you to pass text to the Watson Text to Speech

service and receive audio of the text spoken in a specified language.

[lick here to view code image](#)

```
1 # SimpleLanguageTranslator.py
2 """Use IBM Watson Speech to Text, Language Translator and Text to Spe
3 APIs to enable English and Spanish speakers to communicate."""
4 from watson_developer_cloud import SpeechToTextV1
5 from watson_developer_cloud import LanguageTranslatorV3
6 from watson_developer_cloud import TextToSpeechV1
```

Other Imported Modules

Line 7 imports the `keys.py` file containing your Watson credentials. Lines 8–11 import modules that support this app’s audio-processing capabilities:

- The `pyaudio` module enables us to record audio from the microphone.
- `pydub` and `pydub.playback` modules enable us to load and play audio files.
- The Python Standard Library’s `wave` module enables us to save WAV (Waveform Audio File Format) files. WAV is a popular audio format originally developed by Microsoft and IBM. This app uses the `wave` module to save the recorded audio to a `.wav` file that we send to Watson’s Speech to Text service for transcription.

[lick here to view code image](#)

```
7 import keys # contains your API keys for accessing Watson services
8 import pyaudio # used to record from mic
9 import pydub # used to load a WAV file
10 import pydub.playback # used to play a WAV file
11 import wave # used to save a WAV file
12
```

Main Program: Function `run_translator`

Let’s look at the main part of the program defined in function `run_translator` (lines 13–54), which calls the functions defined later in the script. For discussion purposes, we broke `run_translator` into the 10 steps it performs. In **Step 1** (lines 15–17), we prompt in English for the user to press *Enter*, then speak a question. Function

`record_audio` then records audio for five seconds and stores it in the file `english.wav`:

[lick here to view code image](#)

```
13 def run_translator():
14     """Calls the functions that interact with Watson services."""
15     # Step 1: Prompt for then record English speech into an audio fi
16     input('Press Enter then ask your question in English')
17     record_audio('english.wav')
18
```

In **Step 2**, we call function `speech_to_text`, passing the file `english.wav` for transcription and telling the Speech to Text service to transcribe the text using its *predefined* model `'en-US_BroadbandModel'`.⁴ We then display the transcribed text:

⁴For most languages, the Watson Speech to Text service supports *broadband* and *narrowband* models. Each has to do with the audio quality. For audio captured at 16 kHz and higher, IBM recommends using the broadband models. In this app, we capture the audio at 44.1 kHz.

[lick here to view code image](#)

```
19     # Step 2: Transcribe the English speech to English text
20     english = speech_to_text(
21         file_name='english.wav', model_id='en-US_BroadbandModel')
22     print('English:', english)
23
```

In **Step 3**, we call function `translate`, passing the transcribed text from **Step 2** as the text to translate. Here we tell the Language Translator service to translate the text using its *predefined* model `'en-es'` to translate from English (`en`) to Spanish (`es`). We then display the Spanish translation:

[lick here to view code image](#)

```
24     # Step 3: Translate the English text into Spanish text
25     spanish = translate(text_to_translate=english, model='en-es')
26     print('Spanish:', spanish)
27
```

In **Step 4**, we call function `text_to_speech`, passing the Spanish text from **Step 3** for the Text to Speech service to speak using its voice `'es-US_SofiaVoice'`. We also specify the file in which the audio should be saved:

[lick here to view code image](#)

```
28     # Step 4: Synthesize the Spanish text into Spanish speech
29     text_to_speech(text_to_speak=spanish,    voice_to_use='es-US_SofiaV
30                   file_name='spanish.wav')
31
```

In **Step 5**, we call function `play_audio` to play the file `'spanish.wav'`, which contains the Spanish audio for the text we translated in **Step 3**.

[lick here to view code image](#)

```
32     # Step 5: Play the Spanish audio file
33     play_audio(file_name='spanish.wav')
34
```

Finally, **Steps 6–10** repeat what we did in **Steps 1–5**, but for Spanish speech to English speech:

- **Step 6** records the *Spanish* audio.
- **Step 7** transcribes the Spanish audio to Spanish text using the Speech to Text service's predefined model `'es-ES_BroadbandModel'`.
- **Step 8** translates the Spanish text to English text using the Language Translator Service's `'es-en'` (Spanish-to-English) model.
- **Step 9** creates the English audio using the Text to Speech Service's voice `'en-US_AllisonVoice'`.
- **Step 10** plays the English audio.

[lick here to view code image](#)

```
35     # Step 6: Prompt for then record Spanish speech into an    audio fi
36     input('Press Enter then speak the Spanish answer')
37     record_audio('spanishresponse.wav')
```

```

38
39 # Step 7: Transcribe the Spanish speech to Spanish text
40 spanish = speech_to_text(
41     file_name='spanishresponse.wav', model_id='es-ES_BroadbandModel'
42 )
43 print('Spanish response:', spanish)
44
45 # Step 8: Translate the Spanish text into English text
46 english = translate(text_to_translate=spanish, model='es-en')
47 print('English response:', english)
48
49 # Step 9: Synthesize the English text into English speech
50 text_to_speech(text_to_speak=english,
51     voice_to_use='en-US_AllisonVoice',
52     file_name='englishresponse.wav')
53
54 # Step 10: Play the English audio
55 play_audio(file_name='englishresponse.wav')

```

Now let's implement the functions we call from **Steps 1** through **10**.

Function `speech_to_text`

To access Watson's Speech to Text service, function `speech_to_text` (lines 56–87) creates a `SpeechToTextV1` object named `stt` (short for speech-to-text), passing as the argument the API key you set up earlier. The `with` statement (lines 62–65) opens the audio file specified by the `file_name` parameter and assigns the resulting file object to `audio_file`. The open mode `'rb'` indicates that we'll read (r) binary data (b)—audio files are stored as bytes in binary format. Next, lines 64–65 use the `SpeechToTextV1` object's **recognize method** to invoke the Speech to Text service. The method receives three keyword arguments:

- `audio` is the file (`audio_file`) to pass to the Speech to Text service.
- `content_type` is the media type of the file's contents—`'audio/wav'` indicates that this is an audio file stored in WAV format.⁵

⁵Media types were formerly known as **MIME (Multipurpose Internet Mail Extensions) types**, a standard that specifies data formats, which programs can use to interpret data correctly.

- `model` indicates which spoken language model the service will use to recognize the speech and transcribe it to text. This app uses predefined models—either `'en-US_BroadbandModel'` (for English) or `'es-ES_BroadbandModel'` (for

Spanish).

[lick here to view code image](#)

```
56 def speech_to_text(file_name, model_id):
57     """Use Watson Speech to Text to convert audio file to text."""
58     # create Watson Speech to Text client
59     stt = SpeechToTextV1(iam_apikey=keys.speech_to_text_key)
60
61     # open the audio file
62     with open(file_name, 'rb') as audio_file:
63         # pass the file to Watson for transcription
64         result = stt.recognize(audio=audio_file,
65                               content_type='audio/wav', model=model_id).get_result()
66
67     # Get the 'results' list. This may contain intermediate and final
68     # results, depending on method recognize's arguments. We asked
69     # for only final results, so this list contains one element.
70     results_list = result['results']
71
72     # Get the final speech recognition result--the list's only element
73     speech_recognition_result = results_list[0]
74
75     # Get the 'alternatives' list. This may contain multiple alternative
76     # transcriptions, depending on method recognize's arguments. We
77     # did not ask for alternatives, so this list contains one element.
78     alternatives_list = speech_recognition_result['alternatives']
79
80     # Get the only alternative transcription from alternatives_list.
81     first_alternative = alternatives_list[0]
82
83     # Get the 'transcript' key's value, which contains the audio's
84     # text transcription.
85     transcript = first_alternative['transcript']
86
87     return transcript # return the audio's text transcription
88
```

The `recognize` method returns a `DetailedResponse` object. Its `getResult` method returns a JSON document containing the transcribed text, which we store in `result`. The JSON will look similar to the following but depends on the question you ask:

```

{
  "results": [
    {
      "alternatives": [
        {
          "confidence": 0.983,
          "transcript": "where is the closest bathroom "
        }
      ],
      "final": true
    }
  ],
  "result_index": 0
}

```

Line 70
Line 73
Line 78
Line 81
Line 85

The JSON contains *nested* dictionaries and lists. To simplify navigating this data structure, lines 70–85 use separate small statements to “pick off” one piece at a time until we get the transcribed text—“where is the closest bathroom ”, which we then return. The boxes around portions of the JSON and the line numbers in each box correspond to the statements in lines 70–85. The statements operate as follows:

- Line 70 assigns to `results_list` the list associated with the key 'results':

[lick here to view code image](#)

```
results_list = result['results']
```

Depending on the arguments you pass to method `recognize`, this list may contain intermediate and final results. Intermediate results might be useful, for example, if you were transcribing live audio, such as a newscast. We asked for only final results, so this list contains one element.⁶

⁶For method `recognizes` arguments and JSON response details, see <https://www.ibm.com/watson/developercloud/speech-to-ext/api/v1/python.html?python#recognize-sessionless>.

- Line 73 assigns to `speech_recognition_result` the final speech-recognition result—the only element in `results_list`:

[lick here to view code image](#)

```
speech_recognition_result = results_list[0]
```

- Line 78

[click here to view code image](#)

```
alternatives_list = speech_recognition_result['alternatives']
```

assigns to `alternatives_list` the list associated with the key `'alternatives'`. This list may contain multiple alternative transcriptions, depending on method `recognize`'s arguments. The arguments we passed result in a one-element list.

- Line 81 assigns to `first_alternative` the only element in `alternatives_list`:

[click here to view code image](#)

```
first_alternative = alternatives_list[0]
```

- Line 85 assigns to `transcript` the `'transcript'` key's value, which contains the audio's text transcription:

[click here to view code image](#)

```
transcript = first_alternative['transcript']
```

- Finally, line 87 returns the audio's text transcription.

Lines 70–85 could be replaced with the denser statement

[click here to view code image](#)

```
return result['results'][0]['alternatives'][0]['transcript']
```

but we prefer the separate simpler statements.

Function `translate`

To access the Watson Language Translator service, function `translate` (lines 89–111) first creates a `LanguageTranslatorV3` object named `language_translator`, passing as arguments the service version (`'2018-05-31'`), the API Key you set up earlier and the service's URL. Lines 93–94 use the `LanguageTranslatorV3` object's

translate method to invoke the Language Translator service, passing two keyword arguments:

⁷According to the Language Translator services API reference, '2018-05-31' is the current version string at the time of this writing. IBM changes the version string only if they make API changes that are not backward compatible. Even when they do, the service will respond to your calls using the API version you specify in the version string. For more details, see <https://www.ibm.com/watson-developercloud/language-translator/api/v3/python.html?python#versioning>.

- `text` is the string to translate to another language.
- `model_id` is the predefined model that the Language Translator service will use to understand the original text and translate it into the appropriate language. In this app, `model` will be one of IBM's *predefined* translation models—'en-es' (for English to Spanish) or 'es-en' (for Spanish to English).

[lick here to view code image](#)

```
89 def translate(text_to_translate, model):
90     """Use Watson Language Translator to  translate English to Spanis
91         (en-es) or Spanish to English (es-en) as specified by  model."""
92     # create Watson Translator client
93     language_translator  = LanguageTranslatorV3(version='2018-05-31',
94         iam_apikey=keys.translate_key)
95
96     # perform the translation
97     translated_text  = language_translator.translate(
98         text=text_to_translate,  model_id=model).get_result()
99
100     # Get 'translations' list. If method translate's text  argument
101     # multiple strings, the list will have multiple  entries. We pas
102     # one string, so the list contains only one element.
103     translations_list  = translated_text['translations']
104
105     # get translations_list's only element
106     first_translation  = translations_list[0]
107
108     # get 'translation' key's value, which is the  translated text
109     translation  = first_translation['translation']
110
111     return translation  # return the  translated string
112
```

The method returns a `DetailedResponse`. That object's `getResult` method returns a JSON document, like:

```
{
  "translations": [
    {
      "translation": "¿Dónde está el baño más cercano? "
    }
  ],
  "word_count": 5,
  "character_count": 30
}
```

The JSON you get as a response depends on the question you asked and, again, contains nested dictionaries and lists. Lines 103–109 use small statements to pick off the translated text `"¿Dónde está el baño más cercano? "`. The boxes around portions of the JSON and the line numbers in each box correspond to the statements in lines 103–109. The statements operate as follows:

- Line 103 gets the `'translations'` list:

[lick here to view code image](#)

```
translations_list = translated_text['translations']
```

If method `translate`'s `text` argument has multiple strings, the list will have multiple entries. We passed only one string, so the list contains only one element.

- Line 106 gets `translations_list`'s only element:

[lick here to view code image](#)

```
first_translation = translations_list[0]
```

- Line 109 gets the `'translation'` key's value, which is the translated text:

[lick here to view code image](#)

```
translation = first_translation['translation']
```

- Line 111 returns the translated string.

Lines 103–109 could be replaced with the more concise statement

[lick here to view code image](#)

```
return translated_text['translations'][0]['translation']
```

but again, we prefer the separate simpler statements.

Function `text_to_speech`

To access the Watson Text to Speech service, function `text_to_speech` (lines 113–122) creates a `TextToSpeechV1` object named `tts` (short for text-to-speech), passing as the argument the API key you set up earlier. The `with` statement opens the file specified by `file_name` and associates the file with the name `audio_file`. The mode `'wb'` opens the file for writing (w) in binary (b) format. We'll write into that file the contents of the audio returned by the Speech to Text service.

[lick here to view code image](#)

```
113 def text_to_speech(text_to_speak, voice_to_use, file_name):
114     """Use Watson Text to Speech to convert text to specified voice
115         and save to a WAV file."""
116     # create Text to Speech client
117     tts = TextToSpeechV1(iam_apikey=keys.text_to_speech_key)
118
119     # open file and write the synthesized audio content into the fi
120     with open(file_name, 'wb') as audio_file:
121         audio_file.write(tts.synthesize(text_to_speak,
122                                     accept='audio/wav', voice=voice_to_use).get_result().cont
123
```

Lines 121–122 call two methods. First, we invoke the Speech to Text service by calling the `TextToSpeechV1` object's **`synthesize method`**, passing three arguments:

- `text_to_speak` is the string to speak.
- the keyword argument `accept` is the media type indicating the audio format the Speech to Text service should return—again, `'audio/wav'` indicates an audio file in WAV format.
- the keyword argument `voice` is one of the Speech to Text service's predefined voices. In this app, we'll use `'en-US_AllisonVoice'` to speak English text and

'es-US_SofiaVoice' to speak Spanish text. Watson provides many male and female voices across various languages.⁸

⁸ or a complete list, see

<https://www.ibm.com/watson/developercloud/text-to-speech/api/v1/python.html?python#get-voice>. Try experimenting with other voices.

Watson's `DetailedResponse` contains the spoken text audio file, accessible via `get_result`. We access the returned file's `content` attribute to get the bytes of the audio and pass them to the `audio_file` object's `write` method to output the bytes to a `.wav` file.

Function `record_audio`

The `pyaudio` module enables you to record audio from the microphone. The function `record_audio` (lines 124–154) defines several constants (lines 126–130) used to configure the stream of audio information coming from your computer's microphone. We used the settings from the `pyaudio` module's online documentation:

- `FRAME_RATE`—44100 frames-per-second represents 44.1 kHz, which is common for CD-quality audio.
- `CHUNK`—1024 is the number of frames streamed into the program at a time.
- `FORMAT`—`pyaudio.paInt16` is the size of each frame (in this case, 16-bit or 2-byte integers).
- `CHANNELS`—2 is the number of samples per frame.
- `SECONDS`—5 is the number of seconds for which we'll record audio in this app.

[click here to view code image](#)

```
124 def record_audio(file_name):
125     """Use pyaudio to record 5 seconds of audio to a WAV file."""
126     FRAME_RATE = 44100 # number of frames per second
127     CHUNK = 1024 # number of frames read at a time
128     FORMAT = pyaudio.paInt16 # each frame is a 16-bit (2-byte) integer
129     CHANNELS = 2 # 2 samples per frame
130     SECONDS = 5 # total recording time
131
```

```

132 recorder = pyaudio.PyAudio() # opens/closes audio streams
133
134 # configure and open audio stream for recording (input=True)
135 audio_stream = recorder.open(format=FORMAT, channels=CHANNELS,
136                               rate=FRAME_RATE, input=True, frames_per_buffer=CHUNK)
137 audio_frames = [] # stores raw bytes of mic input
138 print('Recording 5 seconds of audio')
139
140 # read 5 seconds of audio in CHUNK-sized pieces
141 for i in range(0, int(FRAME_RATE * SECONDS / CHUNK)):
142     audio_frames.append(audio_stream.read(CHUNK))
143
144 print('Recording complete')
145 audio_stream.stop_stream() # stop recording
146 audio_stream.close()
147 recorder.terminate() # release underlying resources used by Py
148
149 # save audio_frames to a WAV file
150 with wave.open(file_name, 'wb') as output_file:
151     output_file.setnchannels(CHANNELS)
152     output_file.setsampwidth(recorder.get_sample_size(FORMAT))
153     output_file.setframerate(FRAME_RATE)
154     output_file.writeframes(b''.join(audio_frames))
155

```

line 132 creates the **PyAudio** object from which we'll obtain the input stream to record audio from the microphone. Lines 135–136 use the **PyAudio** object's **open method** to open the input stream, using the constants **FORMAT**, **CHANNELS**, **FRAME_RATE** and **CHUNK** to configure the stream. Setting the **input** keyword argument to **True** indicates that the stream will be used to *receive* audio input. The open method returns a **pyaudio Stream** object for interacting with the stream.

Lines 141–142 use the **Stream** object's **read method** to get 1024 (that is, **CHUNK**) frames at a time from the input stream, which we then append to the **audio_frames** list. To determine the total number of loop iterations required to produce 5 seconds of audio using **CHUNK** frames at a time, we multiply the **FRAME_RATE** by **SECONDS**, then divide the result by **CHUNK**. Once reading is complete, line 145 calls the **Stream** object's **stop_stream method** to terminate recording, line 146 calls the **Stream** object's **close method** to close the **Stream**, and line 147 calls the **PyAudio** object's **terminate method** to release the underlying audio resources that were being used to manage the audio stream.

The **with** statement in lines 150–154 uses the **wave** module's **open** function to open the WAV file specified by **file_name** for writing in binary format ('wb'). Lines 151–153 configure the WAV file's number of channels, sample width (obtained from the

PyAudio object's **get_sample_size method**) and frame rate. Then line 154 writes the audio content to the file. The expression `b''.join(audio_frames)` concatenates all the frames' bytes into a **byte string**. Prepending a string with `b` indicates that it's a string of bytes rather than a string of characters.

Function `play_audio`

To play the audio files returned by Watson's Text to Speech service, we use features of the `pydub` and `pydub.playback` modules. First, from the `pydub` module, line 158 uses the **AudioSegment** class's **from_wav method** to load a WAV file. The method returns a new `AudioSegment` object representing the audio file. To play the `AudioSegment`, line 159 calls the `pydub.playback` module's **play function**, passing the `AudioSegment` as an argument.

[lick here to view code image](#)

```
156 def play_audio(file_name):
157     """Use the pydub module (pip install pydub) to play a WAV file.
158     sound = pydub.AudioSegment.from_wav(file_name)
159     pydub.playback.play(sound)
160
```

Executing the `run_translator` Function

We call the `run_translator` function when you execute `SimpleLanguageTranslator.py` as a script:

[lick here to view code image](#)

```
161 if __name__ == '__main__':
162     run_translator()
```

Hopefully, the fact that we took a divide-and-conquer approach on this substantial case study script made it manageable. Many of the steps matched up nicely with some key Watson services, enabling us to quickly create a powerful mashup application.

13.7 WATSON RESOURCES

IBM provides a wide range of developer resources to help you familiarize yourself with their services and begin using them to build applications.

atson Services Documentation

The Watson Services documentation is at:

<https://console.bluemix.net/developer/watson/documentation>

For each service, there are documentation and API reference links. Each service's documentation typically includes some or all of the following:

- a getting started tutorial.
- a video overview of the service.
- a link to a service demo.
- links to more specific how-to and tutorial documents.
- sample apps.
- additional resources, such as more advanced tutorials, videos, blog posts and more.

Each service's API reference shows all the details of interacting with the service using any of several languages, including Python. Click the **Python** tab to see the Python-specific documentation and corresponding code samples for the Watson Developer Cloud Python SDK. The API reference explains all the options for invoking a given service, the kinds of responses it can return, sample responses, and more.

Watson SDKs

We used the Watson Developer Cloud Python SDK to develop this chapter's script. There are SDKs for many other languages and platforms. The complete list is located at:

<https://console.bluemix.net/developer/watson/sdks-and-tools>

Learning Resources

On the Learning Resources page

<https://console.bluemix.net/developer/watson/learning-resources>

you'll find links to:

- Blog posts on Watson features and how Watson and AI are being used in industry.

- Watson's GitHub repository (developer tools, SDKs and sample code).
- The Watson YouTube channel (discussed below).
- Code patterns, which IBM refers to as "roadmaps for solving complex programming challenges." Some are implemented in Python, but you may still find the other code patterns helpful in designing and implementing your Python apps.

Watson Videos

The Watson YouTube channel

<https://www.youtube.com/user/IBMWatsonSolutions/>

contains hundreds of videos showing you how to use all aspects of Watson. There are also spotlight videos showing how Watson is being used.

IBM Redbooks

The following IBM Redbooks publications cover IBM Cloud and Watson services in detail, helping you develop your Watson skills.

- Essentials of Application Development on IBM Cloud:
<http://www.redbooks.ibm.com/abstracts/sg248374.html>
- Building Cognitive Applications with IBM Watson Services: Volume 1 **Getting Started**: <http://www.redbooks.ibm.com/abstracts/sg248387.html>
- Building Cognitive Applications with IBM Watson Services: Volume 2 **Conversation** (now called Watson Assistant):
<http://www.redbooks.ibm.com/abstracts/sg248394.html>
- Building Cognitive Applications with IBM Watson Services: Volume 3 **Visual Recognition**:
<http://www.redbooks.ibm.com/abstracts/sg248393.html>
- Building Cognitive Applications with IBM Watson Services: Volume 4 **Natural Language Classifier**:
<http://www.redbooks.ibm.com/abstracts/sg248391.html>
- Building Cognitive Applications with IBM Watson Services: Volume 5 **Language Translator**: <http://www.redbooks.ibm.com/abstracts/sg248392.html>

- Building Cognitive Applications with IBM Watson Services: Volume 6 **Speech to Text and Text to Speech:**

<http://www.redbooks.ibm.com/abstracts/sg248388.html>

- Building Cognitive Applications with IBM Watson Services: Volume 7 **Natural Language Understanding:**

<http://www.redbooks.ibm.com/abstracts/sg248398.html>

13.8 WRAP-UP

In this chapter, we introduced IBM's Watson cognitive-computing platform and overviewed its broad range of services. You saw that Watson offers intriguing capabilities that you can integrate into your applications. IBM encourages learning and experimentation via its free Lite tiers. To take advantage of that, you set up an IBM Cloud account. You tried Watson demos to experiment with various services, such as natural language translation, speech-to-text, text-to-speech, natural language understanding, chatbots, analyzing text for tone and visual object recognition in images and video.

You installed the Watson Developer Cloud Python SDK for programmatic access to Watson services from your Python code. In the traveler's companion translation app, we mashed up several Watson services to enable English-only and Spanish-only speakers to communicate easily with one another verbally. We transcribed English and Spanish audio recordings to text, translated the text to the other language, then synthesized English and Spanish audio from the translated text. Finally, we discussed various Watson resources, including documentation, blogs, the Watson GitHub repository, the Watson YouTube channel, code patterns implemented in Python (and other languages) and IBM Redbooks.

14. Machine Learning: Classification, Regression and Clustering

Objectives

In this chapter you'll:

- Use scikit-learn with popular datasets to perform machine learning studies.
- Use Seaborn and Matplotlib to visualize and explore data.
- Perform supervised machine learning with k-nearest neighbors classification and linear regression.
- Perform multi-classification with Digits dataset.
- Divide a dataset into training, test and validation sets.
- Tune model hyperparameters with k-fold cross-validation.
- Measure model performance.
- Display a confusion matrix showing classification prediction hits and misses.
- Perform multiple linear regression with the California Housing dataset.
- Perform dimensionality reduction with PCA and t-SNE on the Iris and Digits datasets to prepare them for two-dimensional visualizations.
- Perform unsupervised machine learning with k-means clustering and the Iris dataset.

Outline

4.1 Introduction to Machine Learning

4.1.1 Scikit-Learn

4.1.2 Types of Machine Learning

4.1.3 Datasets Bundled with Scikit-Learn

4.1.4 Steps in a Typical Data Science Study

4.2 Case Study: Classification with k-Nearest Neighbors and the Digits Dataset, Part 1

4.2.1 k-Nearest Neighbors Algorithm

4.2.2 Loading the Dataset

4.2.3 Visualizing the Data

4.2.4 Splitting the Data for Training and Testing

4.2.5 Creating the Model

4.2.6 Training the Model

4.2.7 Predicting Digit Classes

4.3 Case Study: Classification with k-Nearest Neighbors and the Digits Dataset, Part 2

4.3.1 Metrics for Model Accuracy

4.3.2 K-Fold Cross-Validation

4.3.3 Running Multiple Models to Find the Best One

4.3.4 Hyperparameter Tuning

4.4 Case Study: Time Series and Simple Linear Regression

4.5 Case Study: Multiple Linear Regression with the California Housing Dataset

4.5.1 Loading the Dataset

4.5.2 Exploring the Data with Pandas

4.5.3 Visualizing the Features

4.5.4 Splitting the Data for Training and Testing

4.5.5 Training the Model

4.5.6 Testing the Model

4.5.7 Visualizing the Expected vs. Predicted Prices

4.5.8 Regression Model Metrics

4.5.9 Choosing the Best Model

4.6 Case Study: Unsupervised Machine Learning, Part 1—Dimensionality Reduction

4.7 Case Study: Unsupervised Machine Learning, Part 2—k-Means Clustering

4.7.1 Loading the Iris Dataset

4.7.2 Exploring the Iris Dataset: Descriptive Statistics with Pandas

4.7.3 Visualizing the Dataset with a Seaborn `pairplot`

4.7.4 Using a `KMeans` Estimator

4.7.5 Dimensionality Reduction with Principal Component Analysis

4.7.6 Choosing the Best Clustering Estimator

4.8 Wrap-Up

14.1 INTRODUCTION TO MACHINE LEARNING

In this chapter and the next, we'll present machine learning—one of the most exciting and promising subfields of artificial intelligence. You'll see how to quickly solve challenging and intriguing problems that novices and most experienced programmers probably would not have attempted just a few years ago. Machine learning is a big, complex topic that raises lots of subtle issues. Our goal here is to give you a friendly, hands-on introduction to a few of the simpler machine-learning techniques.

What Is Machine Learning?

Can we really make our machines (that is, our computers) learn? In this and the next

chapter, we'll show exactly how that magic happens. What's the "secret sauce" of this new application-development style? It's data and lots of it. Rather than programming expertise into our applications, we program them to learn from data. We'll present many Python-based code examples that build working machine-learning- models then use them to make remarkably accurate predictions.

Prediction

Wouldn't it be fantastic if you could improve weather forecasting to save lives, minimize injuries and property damage? What if we could improve cancer diagnoses and treatment regimens to save lives, or improve business forecasts to maximize profits and secure people's jobs? What about detecting fraudulent credit-card purchases and insurance claims? How about predicting customer "churn," what prices houses are likely to sell for, ticket sales of new movies, and anticipated revenue of new products and services? How about predicting the best strategies for coaches and players to use to win more games and championships? All of these kinds of predictions are happening today with machine learning.

Machine Learning Applications

Here's a table of some popular machine-learning applications:

Machine learning applications		
Anomaly detection		
Chatbots	Detecting objects in scenes	
Classifying emails as spam or not spam	Detecting patterns in data	Recommender systems ("people who bought this product also bought ")
Classifying news articles as sports, financial, politics, etc.	Diagnostic medicine	
	Facial recognition	Self-Driving cars (more generally, autonomous vehicles)
	Insurance fraud detection	
Computer vision and image		Sentiment analysis (like classifying movie reviews as

classification	Intrusion detection in computer networks	positive, negative or neutral)
Credit-card fraud detection	Handwriting recognition	Spam filtering
Customer churn prediction	Marketing: Divide customers into clusters	Time series predictions like stock-price forecasting and weather forecasting
Data compression	Natural language translation (English to Spanish, French to Japanese, etc.)	Voice recognition
Data exploration		
Data mining social media (like Facebook, Twitter, LinkedIn)	Predict mortgage loan defaults	

14.1.1 Scikit-Learn

We'll use the popular *scikit-learn machine learning library*. Scikit-learn, also called *sklearn*, conveniently packages the most effective machine-learning algorithms as *estimators*. Each is encapsulated, so you don't see the intricate details and heavy mathematics of how these algorithms work. You should feel comfortable with this—you drive your car without knowing the intricate details of how engines, transmissions, braking systems and steering systems work. Think about this the next time you step into an elevator and select your destination floor, or turn on your television and select the program you'd like to watch. Do you really understand the internal workings of your smart phone's hardware and software?

With scikit-learn and a small amount of Python code, you'll create powerful models quickly for analyzing data, extracting insights from the data and most importantly making predictions. You'll use scikit-learn to *train* each model on a subset of your data, then *test* each model on the rest to see how well your model works. Once your models are trained, you'll put them to work making predictions based on data they have not seen. You'll often be amazed at the results. All of a sudden your computer that you've used mostly on rote chores will take on characteristics of intelligence.

Scikit-learn has tools that automate training and testing your models. Although you can

pecify parameters to customize the models and possibly improve their performance, in this chapter, we'll typically use the models' *default parameters*, yet still obtain impressive results. There also are tools like auto-sklearn (<https://automl.github.io/auto-sklearn>), which automates many of the tasks you perform with scikit-learn.

Which Scikit-Learn Estimator Should You Choose for Your Project

It's difficult to know in advance which model(s) will perform best on your data, so you typically try many models and pick the one that performs best. As you'll see, scikit-learn makes this convenient for you. A popular approach is to run many models and pick the best one(s). How do we evaluate which model performed best?

You'll want to experiment with lots of different models on different kinds of datasets. You'll rarely get to know the details of the complex mathematical algorithms in the sklearn estimators, but with experience, you'll become familiar with which algorithms may be best for particular types of datasets and problems. Even with that experience, it's unlikely that you'll be able to intuit the best model for each new dataset. So scikit-learn makes it easy for you to "try 'em all." It takes at most a few lines of code for you to create and use each model. The models report their performance so you can compare the results and pick the model(s) with the best performance.

14.1.2 Types of Machine Learning

We'll present the two main types of machine learning—*supervised machine learning*, which works with *labeled data*, and *unsupervised machine learning*, which works with *unlabeled data*.

If, for example, you're developing a computer vision application to recognize dogs and cats, you'll train your model on lots of dog photos labeled "dog" and cat photos labeled "cat." If your model is effective, when you put it to work processing unlabeled photos it will recognize dogs and cats it has never seen before. The more photos you train with, the greater the chance that your model will accurately predict which new photos are dogs and which are cats. In this era of big data and massive, economical computer power, you should be able to build some pretty accurate models with the techniques you're about to see.

How can looking at unlabeled data be useful? Online booksellers sell lots of books. They record enormous amounts of (unlabeled) book purchase transaction data. They noticed early on that people who bought certain books were likely to purchase other books on the same or similar topics. That led to their *recommendation systems*. Now, when you

rowse a bookseller site for a particular book, you're likely to see recommendations like, "people who bought this book also bought these other books." Recommendation systems are big business today, helping to maximize product sales of all kinds.

Supervised Machine Learning

Supervised machine learning falls into two categories—*classification* and *regression*. You train machine-learning models on datasets that consist of rows and columns. Each row represents a data *sample*. Each column represents a *feature* of that sample. In supervised machine learning, each sample has an associated label called a *target* (like "dog" or "cat"). This is the value you're trying to predict for new data that you present to your models.

Datasets

You'll work with some "toy" datasets, each with a small number of samples with a limited number of features. You'll also work with several richly featured real-world datasets, one containing a few thousand samples and one containing tens of thousands of samples. In the world of big data, datasets commonly have, millions and billions of samples, or even more.

There's an enormous number of free and open datasets available for data science studies. Libraries like scikit-learn package up popular datasets for you to experiment with and provide mechanisms for loading datasets from various repositories (such as `openml.org`). Governments, businesses and other organizations worldwide offer datasets on a vast range of subjects. You'll work with several popular free datasets, using a variety of machine learning techniques.

Classification

We'll use one of the simplest classification algorithms, *k-nearest neighbors*, to analyze the Digits dataset bundled with scikit-learn. Classification algorithms predict the discrete classes (categories) to which samples belong. Binary classification uses two classes, such as "spam" or "not spam" in an email classification application. Multi-classification uses more than two classes, such as the 10 classes, 0 through 9, in the Digits dataset. A classification scheme looking at movie descriptions might try to classify them as "action," "adventure," "fantasy," "romance," "history" and the like.

Regression

Regression models predict a *continuous output*, such as the predicted temperature output in the weather *time series* analysis from [chapter 10's](#) Intro to Data Science

ection. In this chapter, we'll revisit that *simple linear regression* example, this time implementing it using scikit-learn's `LinearRegression` estimator. Next, we use a `LinearRegression` estimator to perform *multiple linear regression* with the California Housing dataset that's bundled with scikit-learn. We'll predict the median house value of a U. S. census block of homes, considering eight features per block, such as the average number of rooms, median house age, average number of bedrooms and median income. The `LinearRegression` estimator, by default, uses *all* the numerical features in a dataset to make more sophisticated predictions than you can with a single-feature simple linear regression.

Unsupervised Machine Learning

Next, we'll introduce unsupervised machine learning with *clustering* algorithms. We'll use *dimensionality reduction* (with scikit-learn's `TSNE` estimator) to *compress* the Digits dataset's 64 features down to two for visualization purposes. This will enable us to see how nicely the Digits data "cluster up." This dataset contains handwritten digits like those the post office's computers must recognize to route each letter to its designated zip code. This is a challenging computer-vision problem, given that each person's handwriting is unique. Yet, we'll build this clustering model with just a few lines of code and achieve impressive results. And we'll do this without having to understand the inner workings of the clustering algorithm. This is the beauty of object-based programming. We'll see this kind of convenient object-based programming again in the next chapter, where we'll build powerful deep learning models using the open source Keras library.

K-Means Clustering and the Iris Dataset

We'll present the simplest unsupervised machine-learning algorithm, *k-means clustering*, and use it on the Iris dataset that's also bundled with scikit-learn. We'll use dimensionality reduction (with scikit-learn's `PCA` estimator) to compress the Iris dataset's four features to two for visualization purposes. We'll show the clustering of the three *Iris* species in the dataset and graph each cluster's *centroid*, which is the cluster's center point. Finally, we'll run multiple clustering estimators to compare their ability to divide the Iris dataset's samples effectively into three clusters.

You normally specify the desired number of clusters, *k*. K-means works through the data trying to divide it into that many clusters. As with many machine learning algorithms, k-means is *iterative* and gradually zeros in on the clusters to match the number you specify.

K-means clustering can find similarities in unlabeled data. This can ultimately help

ith assigning labels to that data so that supervised learning estimators can then process it. Given that it's tedious and error-prone for humans to have to assign labels to unlabeled data, and given that the vast majority of the world's data is unlabeled, unsupervised machine learning is an important tool.

Big Data and Big Computer Processing Power

The amount of data that's available today is already enormous and continues to grow exponentially. The data produced in the world in the last few years equals the amount produced up to that point since the dawn of civilization. We commonly talk about big data, but "big" may not be a strong enough term to describe truly how huge data is getting.

People used to say "I'm drowning in data and I don't know what to do with it." With machine learning, we now say, "Flood me with big data so I can use machine-learning technology to extract insights and make predictions from it."

This is occurring at a time when computing power is *exploding* and computer memory and secondary storage are *exploding* in capacity while costs dramatically decline. All of this enables us to think differently about the solution approaches. We now can program computers to *learn* from data, and lots of it. It's now all about predicting from data.

14.1.3 Datasets Bundled with Scikit-Learn

The following table lists scikit-learn's bundled datasets.¹ It also provides capabilities for loading datasets from other sources, such as the 20,000+ datasets available at `openml.org`.

¹ <http://scikit-learn.org/stable/datasets/index.html>.

Datasets bundled with scikit-learn	
<i>"Toy" datasets</i>	<i>Real-world datasets</i>
Boston house prices	Olivetti faces
Iris plants	20 newsgroups text
Diabetes	Labeled Faces in the Wild face recognition

Optical recognition of handwritten digits	Forest cover types
Linnerrud	RCV1
Wine recognition	Kddcup 99
Breast cancer Wisconsin (diagnostic)	California Housing

14.1.4 Steps in a Typical Data Science Study

We'll perform the steps of a typical machine-learning case study, including:

- loading the dataset
- exploring the data with pandas and visualizations
- transforming your data (converting non-numeric data to numeric data because scikit-learn requires numeric data; in the chapter, we use datasets that are “ready to go,” but we'll discuss the issue again in the “Deep Learning” chapter)
- splitting the data for training and testing
- creating the model
- training and testing the model
- tuning the model and evaluating its accuracy
- making predictions on live data that the model hasn't seen before.

In the “Array-Oriented Programming with NumPy” and “Strings: A Deeper Look” chapters' Intro to Data Science sections, we discussed using pandas to deal with missing and erroneous values. These are important steps in cleaning your data before using it for machine learning.

14.2 CASE STUDY: CLASSIFICATION WITH K-NEAREST NEIGHBORS AND THE DIGITS DATASET, PART 1

o process mail efficiently and route each letter to the correct destination, postal service computers must be able to scan handwritten names, addresses and zip codes and recognize the letters and digits. As you’ll see in this chapter, powerful libraries like scikit-learn enable even novice programmers to make such machine-learning problems manageable. In the next chapter, we’ll use even more powerful computer-vision capabilities when we present the deep learning technology of convolutional neural networks.

Classification Problems

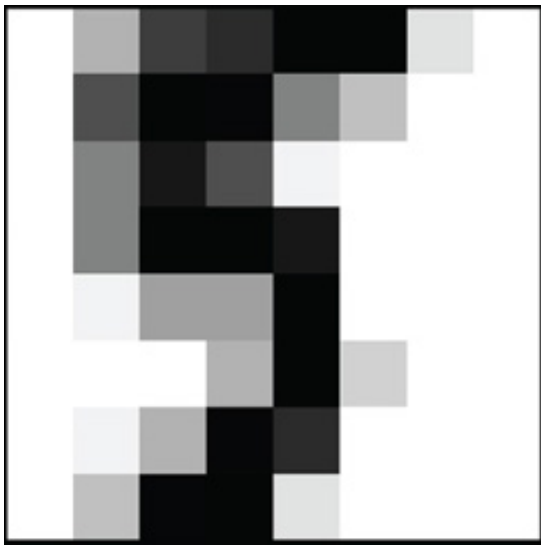
In this section, we’ll look at **classification** in supervised machine learning, which attempts to predict the distinct class ² to which a sample belongs. For example, if you have images of dogs and images of cats, you can classify each image as a “dog” or a “cat.” This is a **binary classification problem** because there are *two* classes.

² Note that the term class in this case means category, not the Python concept of a class.

We’ll use the **Digits dataset** ³ bundled with scikit-learn, which consists of 8-by-8 pixel images representing 1797 hand-written digits (0 through 9). Our goal is to predict which digit an image represents. Since there are 10 possible digits (the classes), this is a **multi-classification problem**. You train a classification model using **labeled data**—we know in advance each digit’s class. In this case study, we’ll use one of the simplest machine-learning classification algorithms, *k-nearest neighbors (k-NN)*, to recognize handwritten digits.

³ <http://scikit-learn.org/stable/datasets/index.html#optical-recognition-of-handwritten-digits-dataset>.

The following low-resolution digit visualization of a 5 was produced with Matplotlib from one digit’s 8-by-8 pixel raw data. We’ll show how to display images like this with Matplotlib momentarily:



Researchers created the images in this dataset from the MNIST database's tens of thousands of 32-by-32 pixel images that were produced in the early 1990s. At today's high-definition camera and scanner resolutions, such images can be captured with much higher resolutions.

Our Approach

We'll cover this case study over two sections. In this section, we'll begin with the basic steps of a machine learning case study:

- Decide the data from which to train a model.
- Load and explore the data.
- Split the data for training and testing.
- Select and build the model.
- Train the model.
- Make predictions.

As you'll see, in scikit-learn each of these steps requires at most a few lines of code. In the next section, we'll

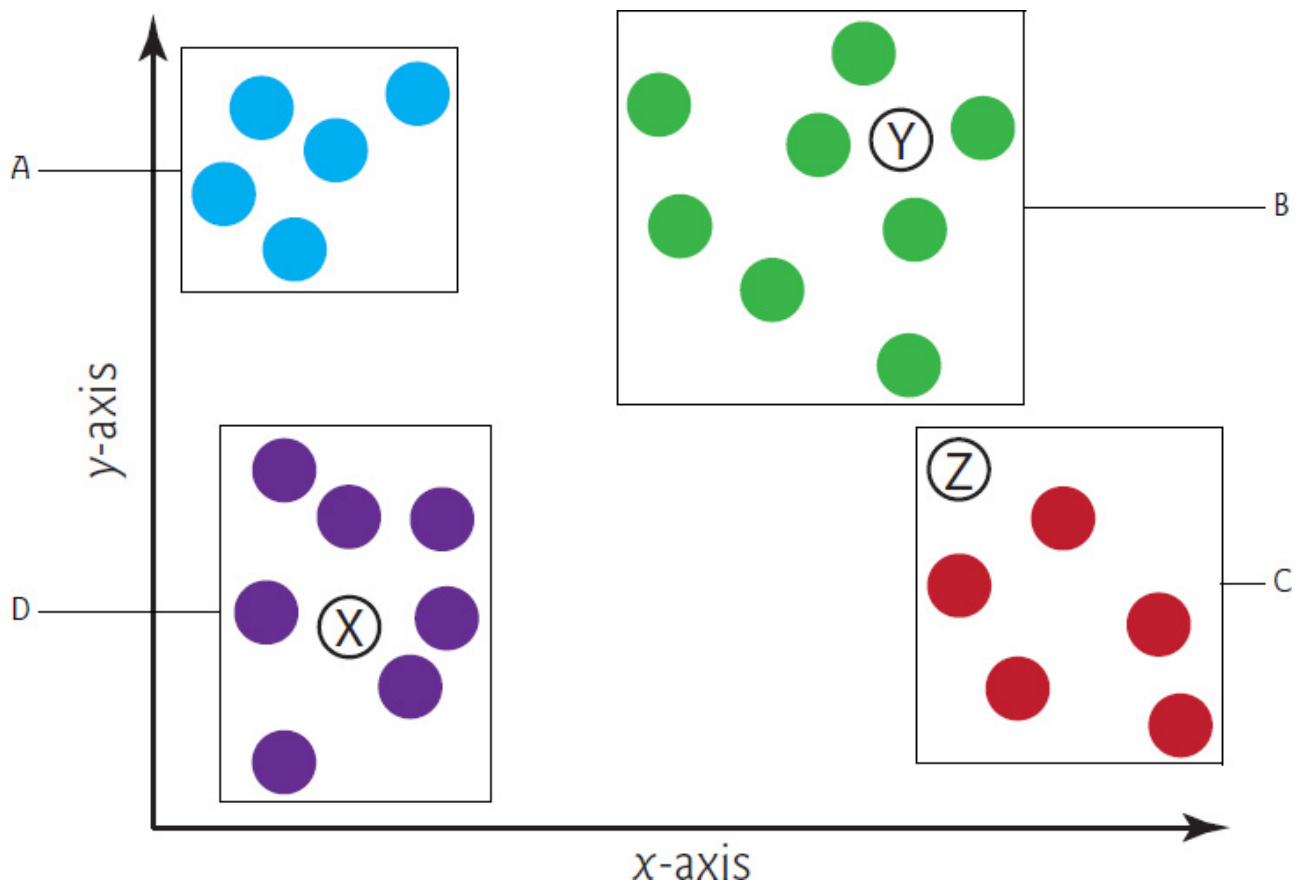
- Evaluate the results.
- Tune the model.
- Run several classification models to choose the best one(s).

We'll visualize the data using Matplotlib and Seaborn, so launch IPython with Matplotlib support:

```
ipython --matplotlib
```

14.2.1 k-Nearest Neighbors Algorithm

Scikit-learn supports many **classification algorithms**, including the simplest—**k-nearest neighbors (k-NN)**. This algorithm attempts to predict a test sample's class by looking at the k training samples that are nearest (in distance) to the test sample. For example, consider the following diagram in which the filled dots represent four sample classes—A, B, C and D. For this discussion, we'll use these letters as the class names:



We want to predict the classes to which the new samples **X**, **Y** and **Z** belong. Let's assume we'd like to make these predictions using each sample's *three* nearest neighbors—*three* is k in the k-nearest neighbors algorithm:

- Sample **X**'s three nearest neighbors are all class D dots, so we'd predict that **X**'s class is D.
- Sample **Y**'s three nearest neighbors are all class B dots, so we'd predict that **Y**'s class is B.

- For **Z**, the choice is not as clear, because it appears *between* the B and C dots. Of the three nearest neighbors, one is class B and two are class C. In the k-nearest neighbors algorithm, the class with the most “votes” wins. So, based on two C votes to one B vote, we’d predict that **Z**’s class is C. Picking an odd *k* value in the kNN algorithm avoids ties by ensuring there’s never an equal number of votes.

Hyperparameters and Hyperparameter Tuning

In machine learning, a **model** implements a machine-learning algorithm. In scikit-learn, models are called **estimators**. There are two parameter types in machine learning:

- those the estimator calculates as it learns from the data you provide and
- those you specify in advance when you create the scikit-learn estimator object that represents the model.

The parameters specified in advance are called **hyperparameters**.

In the k-nearest neighbors algorithm, *k* is a hyperparameter. For simplicity, we’ll use scikit-learn’s *default* hyperparameter values. In real-world machine-learning studies, you’ll want to experiment with different values of *k* to produce the best possible models for your studies. This process is called **hyperparameter tuning**. Later we’ll use hyperparameter tuning to choose the value of *k* that enables the k-nearest neighbors algorithm to make the best predictions for the Digits dataset. Scikit-learn also has *automated* hyperparameter tuning capabilities.

14.2.2 Loading the Dataset

The `load_digits` function from the **sklearn.datasets module** returns a scikit-learn **Bunch** object containing the digits data and information *about* the Digits dataset (called **metadata**):

[lick here to view code image](#)

```
In [1]: from sklearn.datasets import load_digits

In [2]: digits = load_digits()
```

Bunch is a subclass of `dict` that has additional attributes for interacting with the dataset.

Displaying the Description

The Digits dataset bundled with scikit-learn is a subset of the **UCI (University of California Irvine) ML hand-written digits dataset** at:

<http://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits>

The original UCI dataset contains 5620 samples—3823 for training and 1797 for testing. The version of the dataset bundled with scikit-learn contains only the *1797 testing samples*. A Bunch's **DESCR attribute** contains a description of the dataset. According to the Digits dataset's description ⁴, each sample has 64 features (as specified by `Number of Attributes`) that represent an 8-by-8 image with pixel values in the range 0–16 (specified by `Attribute Information`). This dataset has *no missing values* (as specified by `Missing Attribute Values`). The 64 features may seem like a lot, but real-world datasets can sometimes have hundreds, thousands or even millions of features.

⁴ e highlighted some key information in bold.

[lick here to view code image](#)

```
n [3]: print(digits.DESCR)
.. _digits_dataset:

Optical recognition of handwritten digits dataset
-----

**Data Set Characteristics:**

:Number of Instances: 5620
:Number of Attributes: 64
:Attribute Information: 8x8 image of integer pixels in the range
                        0..16.
:Missing Attribute Values: None
:Creator: E. Alpaydin (alpaydin '@' boun.edu.tr)
:Date: July; 1998

This is a copy of the test set of the UCI ML hand-written digits dataset:
http://archive.ics.uci.edu/ml/datasets/
    Optical+Recognition+of+Handwritten+Digits-

The data set contains images of hand-written digits: 10 classes where
each class refers to a digit.

Preprocessing programs made available by NIST were used to extract
normalized bitmaps of handwritten digits from a preprinted form. From a
total of 43 people, 30 contributed to the training set and different 13
to the test set. 32x32 bitmaps are divided into nonoverlapping blocks of
```